

Development of a Marble Racing Simulator

Final Submission

Udbhav Ram

ID: 200008639

ramu@mcmaster.ca

April 7 2026



Department of Physics and Astronomy
McMaster University
Hamilton, Ontario, Canada

Submitted in partial fulfillment of the requirements for the completion of Physics 2VG3

Word Count: 2,500

Abstract

This project is a Panda3D marble-racing demo built on Bullet rigid-body physics. The player controls a spherical marble on a long descending track with banking, rails, boost pads, obstacles, ghost replay, and a bot-race mode. The design goal was to turn a simple rolling-ball simulation into a readable arcade racer by keeping physically believable motion while adding game-specific rules for steering, braking, boosts, finish detection, progression, and feedback. In addition to manual play, the project also explores the possibility of autonomous racing agents as a future extension. The resulting demo shows that Bullet provides a solid base for rolling motion and collisions, but the most playable experience comes from layering game logic on top of raw physics.

Keywords— Physics Engine, Computational Physics, Reinforcement Learning, Genetic Algorithms

Introduction

The core idea is a short-session marble racing game where speed comes from gravity and good line choice rather than from a motorized vehicle. The player's goal is to stay on the track, carry speed through banked turns, use boost pads well, and reach the finish as quickly as possible. The demo supports both Time Trial, where the player chases medals and a saved ghost, and Bot Race, where multiple physical marbles share the same world. The project is intentionally halfway between a simulation and an arcade game. The marble is still a Bullet rigid body, but the experience is shaped through tuned steering forces, soft braking, forgiving finish checks, off-track recovery logic, and visual feedback. That balance is the main design direction of the demo.

How to play

Launch the game using the built-in runner (`./run.sh`), choose a track and mode, then use Left/Right to steer, Down to brake, R to restart, and Space to pause. The goal is to stay on the course, use boost pads cleanly, and cross the finish line as fast as possible; in bot race mode, finish ahead of the other marbles. All code may be accessed at [This Github Repository](#)

Considerations for simulator configuration

A major theme across the project's development was balancing physical accuracy against a fun player experience. Originally, the idea was to create a first-person racing simulator from a marble's perspective on a desk or in the style of a Rube Goldberg machine, resulting in a game that was actually fun to play, not just technically correct in every physical interaction. Early design choices were more dependent on physical accuracy, at least for initial concept validation using a simple ramp, a correctly rotating sphere, Panda3D rendering, and the use of the `sphere.egg` asset from the local 2VG3 repository rather than improvised, imported geometries. From there, the scope was slowly expanded toward a real game with the implementation of a chase camera, keyboard steering via left/right inputs, and a longer desk-style course, more obstacles, and eventually a complete race flow with modes, boosts, ghosts, bots, and even PPO based opponents

This required simplifying overly busy visuals, constraining track generation (and even building out a track map builder as an internal tool) to avoid glitch-prone layouts, adding track-relative logic for off-track resets and finish detection, and making important mechanics such as boosts and starts more immediate and readable. Trivial as it may sound, the process of correctly detecting a reset moment for the player and bots was involved and required some interesting approaches. In the end, a simple coordinate + vertical speed check seemed to do the trick with relatively few, if any, false positives, but this was iterated on many times, including exactly what speed threshold to use. Automatic track generation was considered, with random boost pad and obstacle placement, but ultimately dropped in favour of a single usable and robust track layout with no large aberrations. This was mainly to ensure a robust, playable, and fair racing environment. Similar considerations applied to bot racing in general and eventual RL gym, where inconsistent resets, ambiguous termination, or unstable race-state logic would undermine both fairness and overall game stability. The resulting design, therefore, treats Bullet as the physical foundation for rolling and collisions, while relying on higher-level game rules to enforce valid progression, stable competition, and a more polished arcade-racing experience. Later on, additional quality of life improvements were made to the visualizations and playability of the game, like a live scoreboard, timing, and ghost mode to race against your previous best time in the time trial mode.

Methods

The game was implemented in Python using Panda3D for rendering and scene management and the Bullet library for rigid-body physics. The core simulation models the player and bot marbles as Bullet sphere rigid bodies moving under gravity on a procedurally assembled track composed of ramp segments, rails, boost pads, and obstacle bodies, with parameters such as friction, restitution, steering force, and braking drag tuned for stable rolling and responsive control. Visual assets were primarily constructed in code from simple geometries (spheres and boxes) and generated textures, with optional reuse of local course models where available, and the gameplay layer added race-state logic including off-track resets, finish detection, ghost replay, bot racing, and level loading through JSON-defined track files.

Key physics and modifications

The setup itself is quite simple, as a set of rigid bodies. The player and bot marbles are a dynamic `BulletSphereShape`, and the course, rails, boost pads, and obstacles are static or kinematic box bodies. World-specific gravity is set to 9.81 using `Vec3(0, 0, -9.81)`. The main track is not a mesh collider; it is assembled from many banked rectangular ramp segments, each with its own heading and bank angle, which makes the course stable and predictable in Bullet while still supporting curves and banking in the level builder, and providing stable, discrete segments for the obstacles. `SimulationConfig` is responsible for most of the 'feel' that comes with this game, so in some ways it is tunable (although this is not exposed to the user). The marble itself is deliberately light, at only 0.18 Kg, and has a radius of 0.22. Most ramp sections are at an angle of between 15-17 degrees, with varying coefficients of friction across various elements. Ramp friction 1.18, marble friction 1.05, obstacle friction 0.85, and rail friction 0.9. Restitution is kept very low at 0.02,

which means collisions are intentionally not bouncy; the marble tends to thud and slide/roll rather than pinballing around. Since the player controls are not immediate, this prevents planning your control inputs ahead of time, as opposed to reactive controls for wild bounces. The marble model itself comes with bespoke damping/collision settings. Linear damping is 0.008, and angular damping is 0.010, which keeps the marble from feeling too floaty without killing speed too aggressively. Continuous collision detection is enabled with a tiny motion threshold and a swept sphere radius at 0.98 of the marble radius, which helps prevent phasing through thin geometry at higher speeds. Steering is not done by turning the body directly; instead, the game applies a lateral central force each frame, scaled by marble mass and a configurable steering acceleration of 4.0, so it feels more like a lean than a sharp turn/push. Braking is also not a hard constraint; it is implemented as exponential velocity damping using $\text{BrakeDrag} = 1.2$, which gives smoother speed bleed than clamping velocity directly. These values were tuned primarily through iterative play-testing rather than relying on existing literature or quantitative metrics. As a result, the tuning is quite subjective, it simply felt right to me as a player. That said, there's a reasonable case to be made for sharper, more responsive controls. Boost pads are another deliberate non-physical addition. When the marble overlaps a boost pad, the engine adds forward velocity along the local ramp tangent, rather than applying an impulse from a physical object collision, this caused weird issues with ramp contacting etc. early on, and the current approach worked even if the ball grazed the pad which is the ideal scenario so it gives us one less collision to worry about in the end. It also kept boosts consistent even on banked or curved sections. Similarly, moving obstacles such as pendulums and sweepers are kinematic Bullet bodies, not fully simulated joint systems, as they were in the exercises, although they interact with the marbles in physically correct ways. Their positions are updated every frame from authored motion rules along a prescribed path with a particular radius, and are themselves not affected by collisions with players, which makes them visually and mechanically reliable while still interacting through Bullet contacts and slowing/stopping balls which contact them.

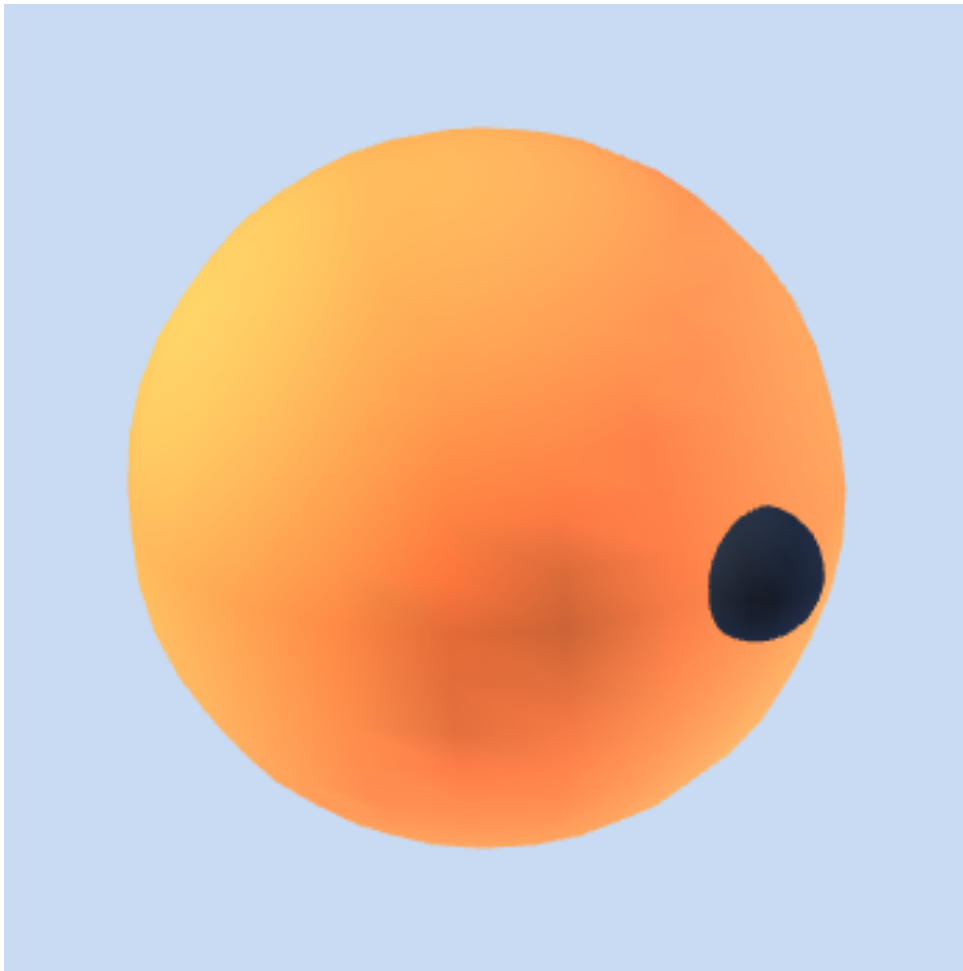


Figure 1: Marble Asset with marker to visually ensure proper rotation

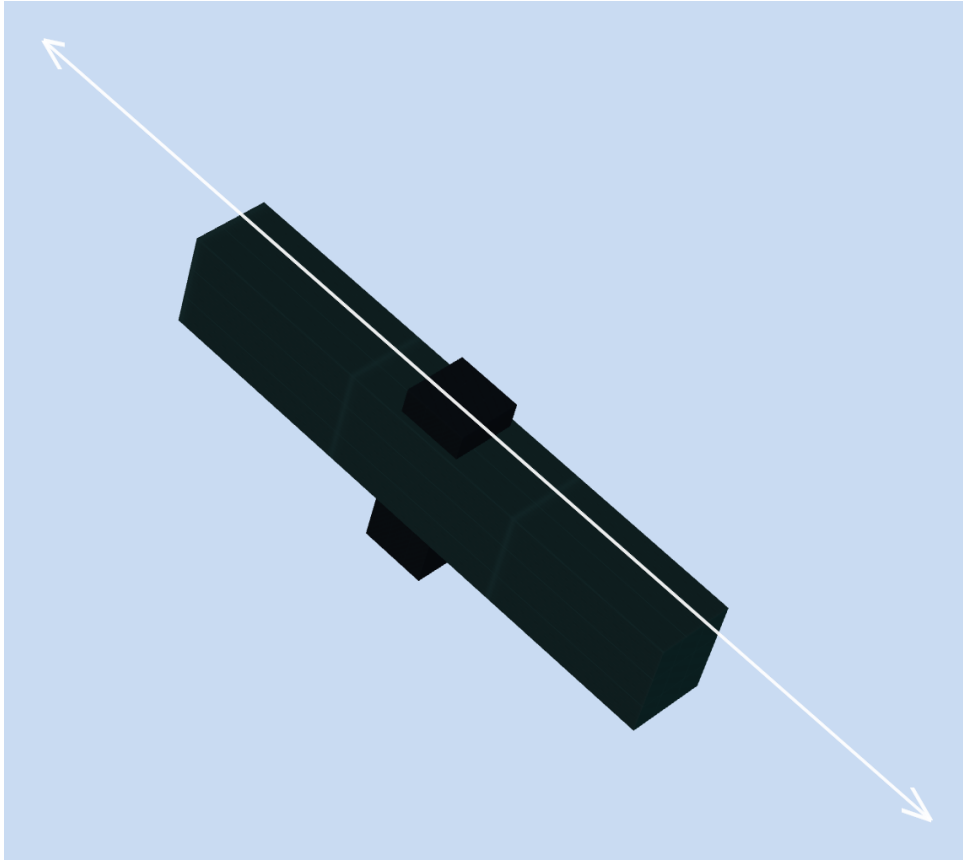


Figure 2: Sweeper Obstacle Asset

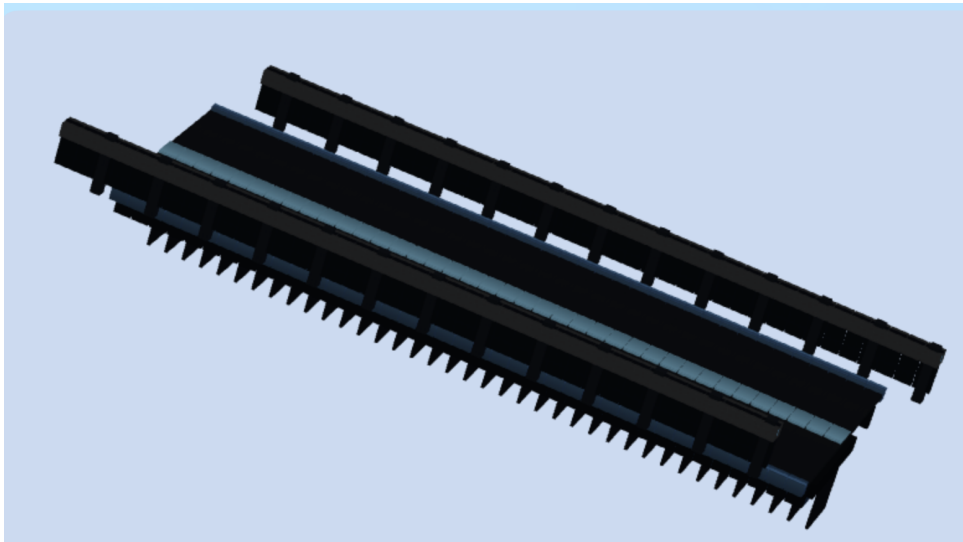


Figure 7: Standard track section

Novel Mechanics

The main novel mechanic is physical marble racing on a banked gravity track. The slope, banking, and collisions all change your line, so movement feels more like managing momentum than steering a normal car in any other racing games. This helped it to keep that rube goldberg like feel, while still being fun to play and controllable. At times, particularly at the start, it does feel slow, and there were some initial pinball spring mechanisms considered to make the start more exciting. Ultimately, it was decided to have the balls 'slingshot' from the beginning with some forward impulse so that the additional momentum-saving dimension of the game was preserved while making the start less boring. The game does have a certain arcade feel with the moving obstacles and the boost pads, which provide a more dynamic and chaotic experience. From the games I've seen, not having a floor was an interesting and unconventional design choice, but it was decided to have a real free-fall experience when the balls are knocked off the track.

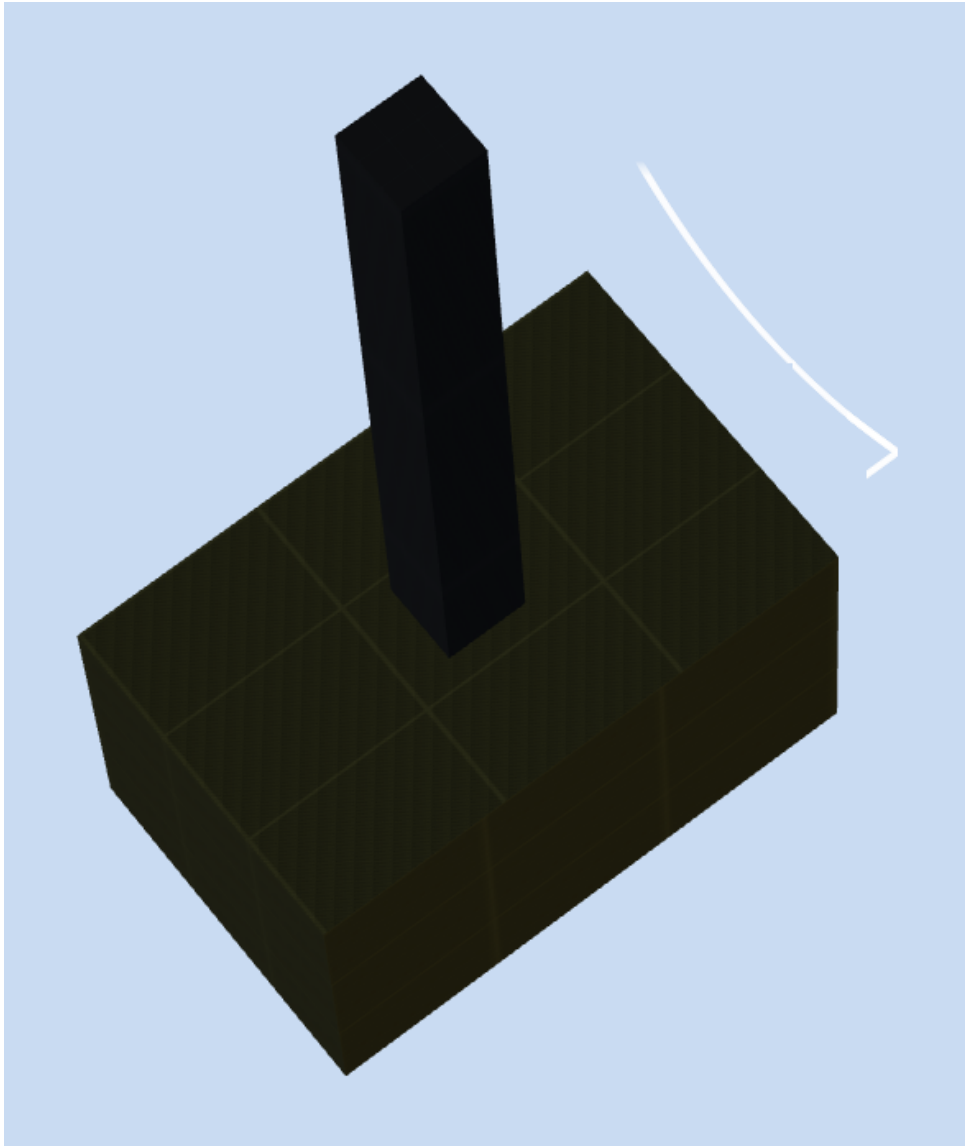


Figure 3: Pendulum Obstacle Asset

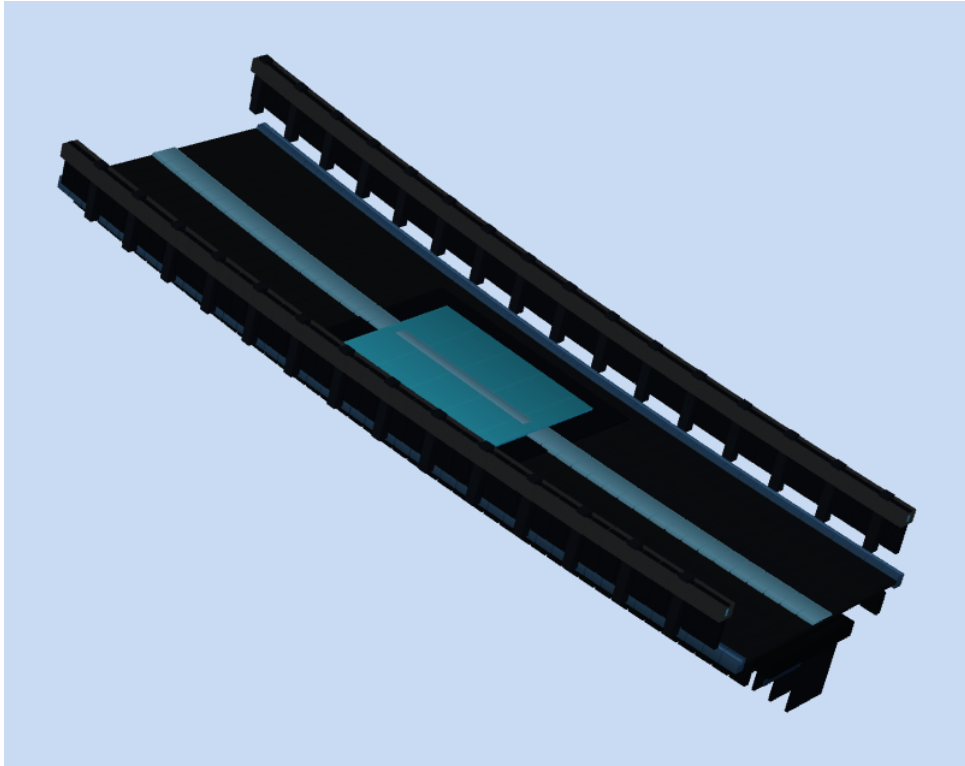


Figure 4: Boost Pad Asset

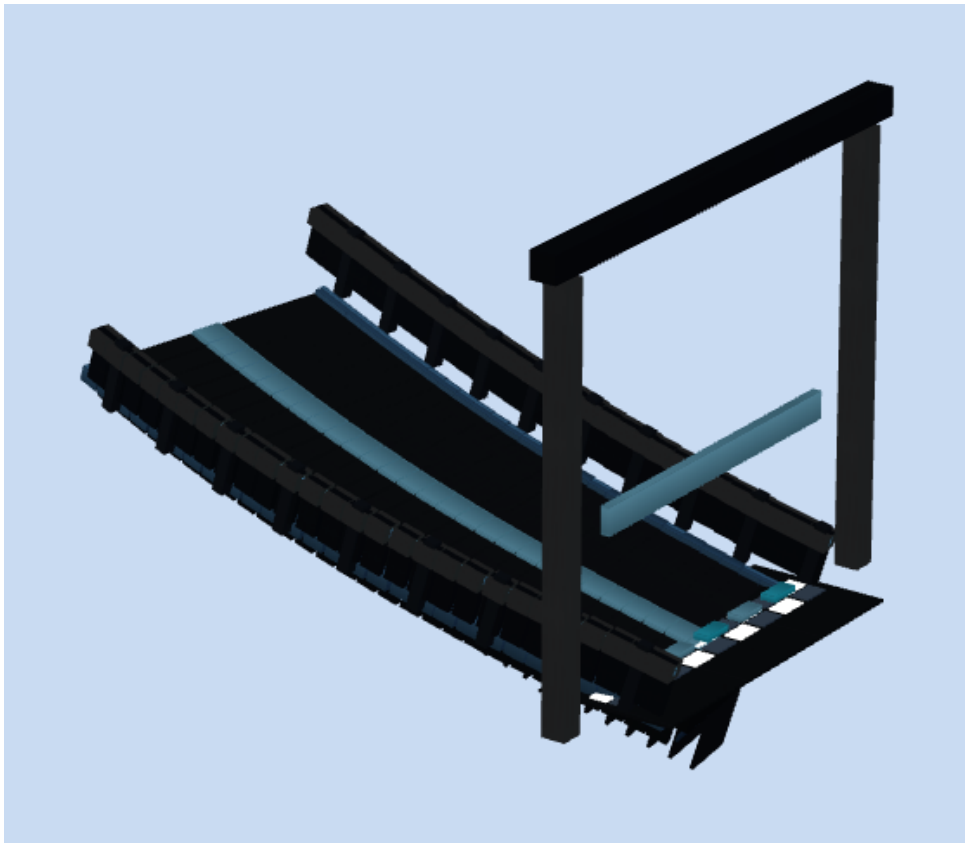


Figure 5: Finish Line Asset

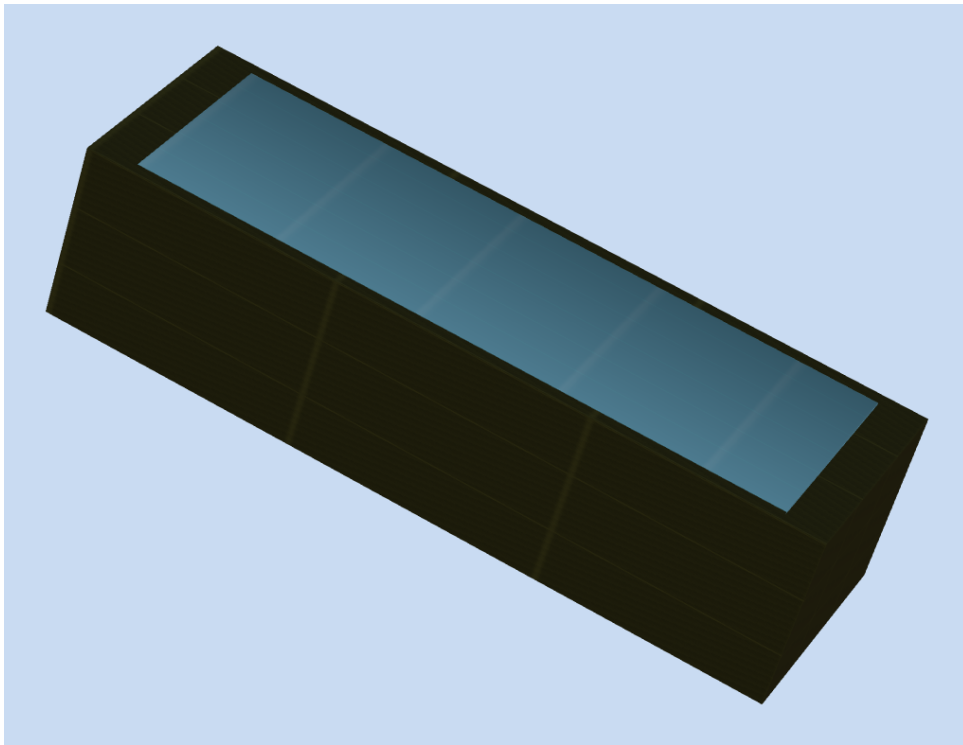


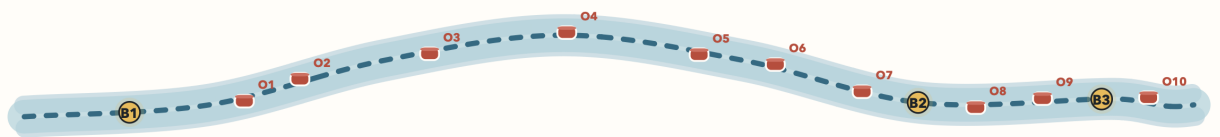
Figure 6: Static Block Obstacle Asset

Figure 2. Gameplay elements on the authored course

Boost pads, obstacle bodies, and the nominal center line are shown in track coordinates.

Default level footprint

Outer rails are implied by the pale boundary lines; the dashed centerline is only a guide for the report.



● Boost pads
 ■ Obstacles
 --- Center racing line

3 authored boosts
 10 authored obstacles
 836 Bullet ramp segments

Figure 8: Course Layout

Figure 1. Default track geometry

Course footprint paired with elevation, banking, and heading extracted directly from the authored level.

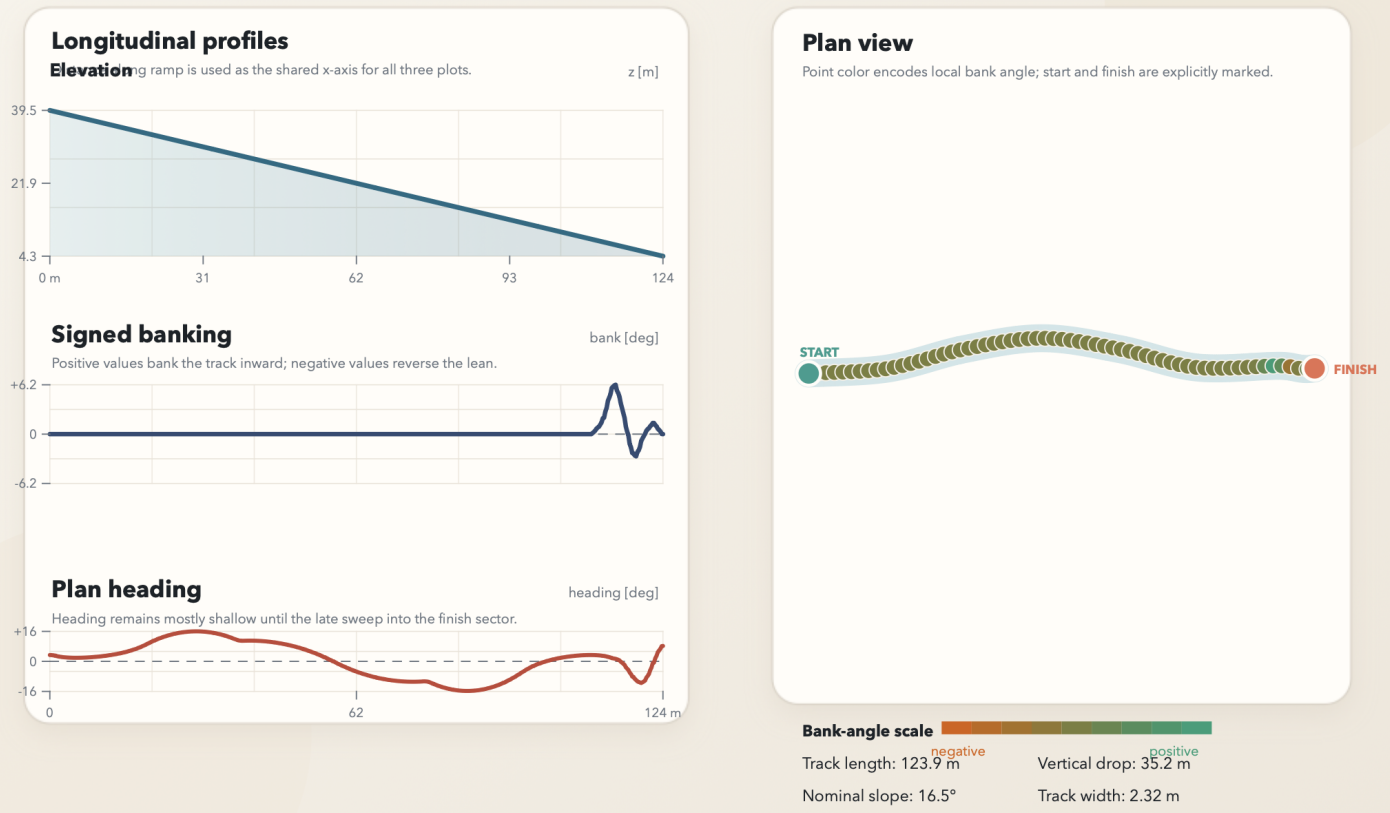


Figure 9: Additional Track Metrics

Further Iteration

The next steps are fairly clear: implementing more tracks, multiplayer (LAN or online), upgrades to balls, unlocking new areas, and special abilities such as jumps or temporary boosts. Another promising direction is autonomous bot racing, detailed below.

Technical note on reinforcement learning As a supplementary extension, I also explored autonomous bot racing using a PPO-based direct-control policy. The training setup used a 23-dimensional normalized observation vector describing progress and velocity, lane and contact state, boost and obstacle context, and short-horizon track geometry. The policy itself was a compact multilayer perceptron with architecture 23-64-64-2, where the two outputs controlled steering and braking directly.

Training followed a two-stage curriculum. First, the policy was trained on a short obstacle-free time-trial course to stabilize basic motion. It was then trained on the full default course with boosts and moving hazards, with spawn positions reset at multiple distances along the track so that the agent repeatedly practiced later-course sections instead of only relearning the opening ramp. Checkpoints were evaluated every 100k timesteps.

The recorded late-stage training results were encouraging. The checkpoint slice reached 2,924,544 total timesteps and included 11 evaluated checkpoints, 7 of which successfully finished the course. The fastest successful finish was 31.37 s, while the best successful checkpoint was generation 27 with reward 116.63 and a finish time of 34.92 s. These results suggest that the simulator is already stable enough to support learning-based control in addition to manual play.

Critiques

The biggest drawback, as I see it, is just how much additional custom logic is required to deal with edge cases. The raw rigid-body simulation alone is not enough to make the game feel fair and even, and to produce this MVP demo required a lot of deep thought about clever game design. Bullet also isn't the most conducive to video game physics out of the box; it seems like it would be great for first-year kinematic labs simulations, but something like Unity, which is specifically tuned to work with assets and video game creation, would make this overall better. The biggest feature that would be great to have would be support for controlled rolling and contact reasoning, stable sphere-on-track behaviour at speed, better ways to classify "supported vs falling vs scraping," and more expressive constraints for hazards like true pendulums. Mixing the animations of the pendulums and sweepers also required a lot of thought, so a hybrid mode where animations could be applied to simulated objects would be great.



Figure 10: Sample screenshot of best generation playback compared to heuristic bot

Reality vs. Game Physics

As we’ve seen, we’ve already had to make several changes away from pure physics in order to make the game more playable, let alone enjoyable. The tweaks in the simulator config were important not only for playability but also to provide a sufficiently robust training gym for the RL agents, so it’s something that was developed very iteratively and with a lot of thought and testing.

Conclusions

The marble racing simulator successfully converts a Bullet-based rigid-body simulation into a fully playable arcade racer. Using Panda3D and Bullet for gravity-driven rolling, banked tracks, collisions, and kinematic obstacles, the project demonstrates that a general-purpose physics engine can deliver engaging gameplay when augmented with targeted game logic. Features such as boost pads, ghost replays, off-track recovery, and RL trained bot opponents all share the same physics world, providing consistent interactions and responsive control.

The final demo achieves all of the original goals, wherein the marble remains a true Bullet sphere under realistic gravity and friction, yet the experience feels polished and fun through custom steering forces, exponential braking, non-physical boosts, and forgiving race rules. Supplementary PPO-based bot training further suggests that the simulator is stable enough to support learning-based control, although the exported learned policy is not yet fully integrated into the current runtime bot path.

A key insight from iterative tuning is that Bullet excels at rolling and collision mechanics, but the most enjoyable player experience arises from layering higher-level game rules on top of raw physics. This hybrid approach mirrors commercial game development and highlights the value of computational physics for interactive applications.

In summary, the project validates Panda3D and Bullet as practical tools for student-led physics-based games. With stable core systems in place, the simulator now serves as a solid foundation for future enhancements, including additional tracks, multiplayer, improved agents, or mobile adaptations.